

Visum

AI Agent Readiness Scanner

Project documentation: architecture, current state, what works, what does not, and what needs your decision.

Generated 26 July 2026 | branch main | 105 commits

Production URL	visum-eight.vercel.app
Scanner backend	visum-backend.onrender.com
Database	Supabase (Postgres), project hzdmfyesfvjcrtztixo
Repository	github.com/UTSAV-2024/visum

Read this first

Two things are waiting on you and nothing else is blocked by them: **Stripe** needs its products and keys before card checkout switches on, and **AI-crawler verification** is the honest next step for AI Analytics. Everything else described here is live and verified in production.

1. What Visum does

Visum scans any public website for **AI-agent readiness** - how well AI systems such as ChatGPT, Claude and Perplexity can find, read and cite it - and returns a 0-100 score with per-check findings and prioritised fixes in roughly 20 seconds.

The distinction from traditional SEO matters: Visum measures the technical signals that decide whether an AI agent can consume a page at all, not how it ranks for a human searcher.

The eight checks

Check	Weight	What it measures
JSON-LD Structured Data	20	Whether AI can understand what the page is about without guessing
AI Bot Permissions (robots.txt)	15	Whether AI crawlers are permitted. No robots.txt scores 13/15, not 0 - absence means allowed
llms.txt File	10	Presence and completeness of an llms.txt summary
MCP Endpoint	10	A Model Context Protocol endpoint or OpenAPI spec. Emerging, so framed as a bonus
JavaScript Rendering	10	How much content survives without JS. Most AI crawlers do not run it
Meta Tags and Open Graph	10	Title, description and OG tags used for page context
Page Load Speed	10	Real TTFB and load time via the Navigation Timing API
Sitemap.xml	5	Whether crawlers can discover the whole site

Scores are normalised to /100. A check that cannot be measured is **excluded** from the score and disclosed in the headline, rather than being silently scored zero.

2. Architecture

Layer	Stack	Hosting
Frontend	Next.js 16 (Pages Router), React 19, Tailwind v4	Vercel
Backend	FastAPI, Playwright (Chromium), httpx	Render (Docker)
Database + Auth	Supabase Postgres, Row Level Security	Supabase Cloud
Payments	Stripe Checkout + webhook (built, not switched on)	Stripe
Analytics	PostHog, Vercel Analytics	-

Request flow for a scan

1. Browser posts a URL to `/api/scan` on the frontend. The browser never talks to the scanner backend directly.
2. The route verifies the session, then claims one scan from quota under a row lock *before* scanning, so two concurrent scans cannot spend the same last credit.
3. It proxies to the FastAPI backend, which crawls and runs the eight checks.
4. The result is stored against the user; if the scan failed, the quota claim is handed back.

3. Application routes

Every route below renders the signed-in user's own data. There is no sample data and no preview banner anywhere in the product.

Route	Purpose
/dashboard	Score, scan history, recently fixed items - all derived from the user's scans
/analytics	AI crawler traffic reported by the customer's own server (see section 6)
/recommendations	Prioritised fixes derived from the latest scan's failed checks
/competitors	Scan rivals and compare check by check. Costs a scan credit
/reports	Score and issue trends across scan history
/team	Members, roles, invite links, shared quota pool
/result	The report for a scan just run. Reads sessionStorage, so it is reached from a scan, not the sidebar
Marketing and auth pages: / /about /pricing /contact /privacy /terms /login /signup /join/[token]	

4. API endpoints

Endpoint	Auth	Notes
POST /api/scan	Session	The only way to run a scan. Enforces quota server-side
POST /api/competitors/scan	Session	Same rules; stored as kind='competitor' so it never pollutes the dashboard
GET /api/account	Session	Plan, quota, storage. The source for the usage meters
GET POST DELETE /api/analytics/sites	Session	Register a domain for crawler tracking; returns its ingest key
GET /api/analytics/bots	Session	Crawler counts, timeline and top paths
POST /api/collect	Ingest key	Called by the customer's server. Append-only, cannot read anything
POST /api/subscription/checkout	Session	Creates a Stripe session. Grants nothing on its own
POST /api/subscription/webhook	Stripe signature	The only path that grants a paid plan
POST /api/subscription/upgrade	Session	Manual grant. Refuses unless ALLOW_UNVERIFIED_UPGRADES=true
GET POST /api/mcp	None	Visum's own MCP endpoint, so AI agents can scan sites. Dogfoods the MCP check
GET /api/auth/callback	None	OAuth code exchange; establishes the session server-side

5. Data model

Row Level Security is on for every table. The browser can **read** its own rows and nothing else; every write that matters goes through the service role on the server. This is what makes quota untamperable - a signed-in user cannot raise their own scan limit, zero their counter, insert a payment, or forge a scan row.

Table	Holds
scans	One row per scan: url, score, band, per-check JSON, timing, storage cost, kind, org
profiles	Name, email, avatar - populated from the auth provider
subscriptions	Tier, status, scan limit, storage limit, renewal date, provider ids
usage	Scans used in the current period
payments	Revenue ledger. Also the record of 'already fulfilled' for webhook replays
storage_usage	View. Storage is summed from scans, so it cannot drift from reality
organizations, organization_members, organization_invites	Teams, roles and invite links. Members draw on the owner's shared quota pool
tracked_sites	A domain registered for crawler tracking, with its ingest key
bot_visits	One row per AI crawler request. No raw IP - a salted hash only

Table	Holds
events	Visum's own product analytics. Not customer traffic

6. AI Analytics and crawler tracking

This page previously showed entirely invented numbers. It now reports only what a customer's own server measured.

Why collection is server-side, not a tracking pixel

AI crawlers largely do not execute JavaScript - which is precisely what Visum's own JavaScript Rendering check measures. A browser pixel would therefore have recorded almost none of this traffic, and an image beacon fares no better because most crawlers do not fetch sub-resources. Reporting happens from the customer's server: Next.js middleware, a Cloudflare Worker, or a plain batched HTTP call.

How it works

The customer registers a domain and receives an **ingest key**. Their server filters requests to AI crawler user-agents and posts them to `/api/collect`. Ordinary visitor traffic never leaves their infrastructure - the snippet filters before it calls out, and the endpoint discards anything that is not a recognised crawler.

23 crawlers are recognised, labelled by vendor and by **purpose** - training, search, or user-triggered. The distinction matters: blocking a training crawler is a choice, whereas blocking a retrieval crawler is what makes a site invisible in AI answers.

Two deliberate honesty constraints

Every row is marked unverified. A user-agent is a claim, not proof - anything can call itself GPTBot. Rows only become verified once the source IP is checked against the vendor's published ranges. The page says which it is rather than presenting spoofable counts as fact. *This verification is not built yet.*

'Nothing reported yet' is distinguished from 'zero crawls'. These are different facts and the page never shows 0 for the first.

7. Accounts, quota and billing

Plan	Price	Scans	Notes
Free	\$0	3 lifetime	Not a recurring allowance - it is for seeing what Visum finds
Pro	\$15/mo	30 per week	Recommended tier
Ultimate	\$70/mo	100 per week	

Plan numbers live in one place - `frontend/lib/plans.js` - and the pricing page copy is interpolated from it, so the page cannot advertise a limit the server will not provision.

Stripe status: built, not switched on

Checkout creates a session and grants nothing. The plan is provisioned only when Stripe confirms payment in the webhook. That split is what stops a replayed request from buying a plan for free.

Two non-obvious correctness points were handled:

- **Raw body required.** Stripe signs the exact bytes; Next's JSON parser reserialises them, so signature verification would have failed every time. The body parser is off.
- **Idempotency.** Stripe delivers at least once. Re-running provisioning resets the scan counter to zero, so a duplicate delivery would have handed out a free allowance and double-counted revenue. The ledger row is checked first.

Verified locally: forged signature rejected 400, missing signature 400, valid signature 200, and 501 with no side effects while unconfigured. **Untested against real Stripe traffic** until the keys exist.

8. Known issues and limitations

Item	Status	Detail
Crawler verification	Open	All bot_visits rows are unverified. Needs IP checks against vendor ranges
Stripe not live	Needs you	Products, prices and keys required before checkout activates
Payments race	Open	The webhook replay guard is a read-then-write. A unique index on payments(provider_payment_id) would close it at the database level
Service key rotation	Needs you	The Supabase service-role key has not been rotated. Dashboard action
Local dev preview	Environmental	On this Windows machine the local production build does not hydrate and 'next dev' has an HMR crash loop. Production is unaffected - verify interactive behaviour against the deploy, not locally
Two Python venvs	Housekeeping	A root venv/ and backend/venv/ both exist. Both gitignored

9. Hard-won operational lessons

Each of these cost real debugging time. They are recorded because none of them is obvious from the code.

A Render service's runtime cannot be changed in place

Playwright's Chromium needs system libraries that Render's native Python runtime cannot install without root, so two of the eight checks silently reported 'not measured' on every production scan. The Docker fix sat in the repo for days doing nothing: pushing `render.yaml` does not convert an existing service to Docker. The

service had to be replaced via a Blueprint sync.

Related: a Blueprint with no plan field defaults to a **paid** tier. It is now pinned to free explicitly.

NEXT_PUBLIC_ variables are inlined at build time

Changing one has no effect on a deployment that was already built. Twice during this work a fix appeared not to apply because the variable changed *after* the build that shipped it. Always redeploy after the change, with the build cache off.

Related: an environment variable **overrides** a code fallback. Centralising a URL in code does nothing if Vercel still sets the old value.

Rebuilding an old deployment promotes old code

In the Vercel deployment list, redeploying a row labelled 'Production rebuild of ...' re-promotes whatever commit that older deployment was built from. This silently reverted a fix that had been live for about a minute. Redeploy the row showing the branch and commit you actually want.

Content-Security-Policy silently broke all authentication

The CSP's connect-src never listed the Supabase host, so the browser blocked every auth request and users saw a bare 'Failed to fetch'. It looked like a broken server; it was a policy line. Any new third-party host the browser must reach has to be added there.

10. Environment variables

Frontend (Vercel)

Variable	Required	Purpose
NEXT_PUBLIC_SUPABASE_URL	Yes	Auth and data. Public by design
NEXT_PUBLIC_SUPABASE_ANON_KEY	Yes	Public; RLS is what protects data
SUPABASE_URL	Yes	Server-side reads and writes
SUPABASE_SERVICE_KEY	Yes	Bypasses RLS. Server only - never NEXT_PUBLIC_
NEXT_PUBLIC_API_URL	Yes	Scanner backend. Overrides the code fallback
STRIPE_SECRET_KEY	For payments	Signs checkout sessions
STRIPE_WEBHOOK_SECRET	For payments	Without it, checkout stays off by design
STRIPE_PRICE_PRO / _ULTIMATE	For payments	Recurring price ids. Must match plans.js
BOT_IP_SALT	Recommended	Salts crawler IP hashes. Falls back to the service key
ALLOW_UNVERIFIED_UPGRADES	No	Manual plan grants. Never set in production

Backend (Render)

All optional with sensible defaults. DATABASE_URL is currently unset - scan history lives in Supabase via the frontend. MAX_CONCURRENT_SCANS is pinned to 1 so a single Chromium fits in the free tier's 512 MB.

11. Running and deploying

Local

```
cd frontend && npm install && npm run dev
cd backend && uvicorn app.main:app --reload --port 8000
docker compose up # both together
```

Note the local preview caveat in section 8: verify interactive UI against the deployed site, not the local build. `npm run build` is still the first gate - it type-checks.

Deploying

- **Frontend:** merge to main; Vercel deploys automatically. If you changed an environment variable, redeploy afterwards with the cache off.
- **Backend:** Blueprint-managed from `render.yaml`; auto-deploys on push to main.
- **Database:** `frontend/supabase-migration.sql` mirrors the live schema and is idempotent - safe to re-run.

12. What to do next

#	Task	Who
1	Create Stripe products and prices (Pro \$15/mo, Ultimate \$70/mo recurring), set the four STRIPE_ variables, redeploy, then point a webhook at <code>/api/subscription/webhook</code>	You

#	Task	Who
2	Manually confirm the sign-in and sign-up screens read the way you want - the fix is verified at the network layer but the form could not be driven by tooling	You
3	Rotate the Supabase service-role key	You
4	Build AI crawler IP verification, so bot_visits rows can be trusted rather than merely claimed	Claude
5	Add the unique index closing the webhook replay race	Claude
6	Decide whether crawler tracking should be a paid-tier feature, and whether AI visibility testing (querying engines for citations) is worth its per-check API cost	Decision

A note on what this product now claims

Six of twelve navigation destinations previously rendered invented data behind a 'preview' banner. They were removed rather than built, because each needed a data source that does not exist and a banner does not repair the trust damage of clicking half a product into sample data. Roughly 10,000 lines were deleted. What remains is smaller and entirely real - which is the version worth selling.